

PDF to Markdown Pipeline

Full Project Description, Architecture, Schemes, Tools, and Build Plan

Local-first, Docker-based, artifact-driven document processing platform

Document type:	Technical architecture specification
Primary goal:	Build a complete product skeleton that can grow from placeholders into a production system
Main input:	PDF document
Main outputs:	Markdown, structured JSON, and full intermediate artifacts
Runtime model:	Multiple Docker containers, one service per responsibility
Communication:	FastAPI over internal Docker network for phase 1
Version target:	Phase-1 / Phase-2 project planning document

April 20, 2026

Contents

1	Executive Summary	2
2	What the Product Must Do	2
3	What the First Version Must Actually Do	3
4	Core Architectural Principles	3
4.1	One Service = One Responsibility	3
4.2	One Service = One Docker Container	3
4.3	Stable Contracts Between Services	4
4.4	Artifact-Driven Design	4
5	High-Level System Scheme	4
6	Detailed Description of Each Service	5
6.1	1. Orchestrator	5
6.2	2. Page Preparation Service	5
6.3	3. Layout Master	6
6.4	4. Block Router	6
6.5	5. Text OCR	7
6.6	6. Formula OCR	7
6.7	7. Table OCR	7
6.8	8. Image Classifier	8
6.9	9. Image OCR / Figure Worker	8
6.10	10. Reading Order Service	8
6.11	11. Assembly Service	8
6.12	12. Validation Service	9
7	Communication Between Containers	9
7.1	Important Concept	9
7.2	Internal Addressing	9
7.3	Recommended Minimal Endpoints Per Worker	9
8	Project Repository Structure	10
9	Service Internal Structure	11
10	Artifact Contract	11
10.1	Why This Contract Is Important	11
11	Job State Model	12
12	Docker and Configuration Model	12
12.1	Why We Still Need Environment Variables	12
12.2	Recommended Files	13
12.3	Example .env Values	13
13	Docker Compose Strategy	13
14	Python, Virtual Environments, and Containers	13
15	Git and GitHub Strategy	14
15.1	What Must Be Versioned	14

15.2 What Must Not Be Versioned	14
15.3 Recommended .gitignore Sections	14
16 Large Models and Multi-Machine Work	15
16.1 What GitHub Is For	15
16.2 What GitHub Is Not For	15
16.3 Correct Strategy	15
16.4 Per-Machine Example	15
16.5 Recommended Operational Pattern	16
16.6 Model Setup Scripts	16
17 Enumeration of Main Instruments and Technologies	16
17.1 Infrastructure and Packaging	16
17.2 Programming and API Layer	17
17.3 Shared Engineering Tools	17
17.4 Possible Document / Vision Engines	17
18 Suggested API Contracts	17
18.1 Health	17
18.2 Capabilities	17
18.3 Process	18
19 Suggested Shared Schemas	18
20 Ready Build Plan	18
20.1 Stage 0: Create the Repository	18
20.2 Stage 1: Create Placeholder Services	18
20.3 Stage 2: Create the Orchestrator	19
20.4 Stage 3: Add docker-compose	19
20.5 Stage 4: Add Shared Schemas	19
20.6 Stage 5: Add Basic Integration Tests	19
20.7 Stage 6: Replace Placeholders One by One	20
20.8 Stage 7: Production Hardening Later	20
21 Recommended Reading Order for Building the Project	20
22 Final Architecture Summary	20
23 Concise Master Scheme	21

1. Executive Summary

This project is a modular platform that receives a PDF document and produces a Markdown representation together with a structured JSON representation and a full set of intermediate artifacts. The system is intentionally designed as a **pipeline of Docker containers**, not as one large monolithic application.

The key architectural idea is simple:

- one service receives and coordinates jobs;
- one service prepares pages from the PDF;
- one service detects layout blocks;
- one service routes blocks to specialized workers;
- different workers process text, formulas, tables, and images;
- one service determines reading order;
- one service assembles final Markdown and JSON;
- one service validates the final result.

This makes the product easier to build, easier to test, easier to debug, and much easier to extend when different OCR or vision models are introduced later.

2. What the Product Must Do

The product must eventually support the following end-to-end behavior:

1. Receive a PDF from a client.
2. Create a job id and a job folder.
3. Store the original document and job metadata.
4. Convert the PDF into page images.
5. Detect layout blocks on each page.
6. Route blocks to the correct worker.
7. Extract content from each block:
 - text blocks;
 - formulas;
 - tables;
 - figures and image-like regions.
8. Reconstruct logical reading order.
9. Assemble a final Markdown file and structured JSON file.
10. Validate results and mark the job as completed, degraded, or failed.
11. Keep all intermediate artifacts so the whole process can be inspected later.

3. What the First Version Must Actually Do

The first version does **not** need perfect OCR or advanced models. The first version needs a **correct skeleton**.

That means phase 1 should provide:

- a clean monorepo;
- one Dockerfile per service;
- a working docker-compose setup;
- a hello-world orchestrator;
- placeholder services for all major pipeline stages;
- stable request/response schemas;
- a job folder layout;
- a clear Git and model-storage strategy.

The point is to prove the **architecture**, not model accuracy.

4. Core Architectural Principles

4.1 One Service = One Responsibility

Each service should do one thing well.

Examples:

- the layout service finds blocks;
- the text OCR service reads text regions;
- the formula OCR service reads formulas;
- the table OCR service reads tables;
- the image classifier labels figure-like regions;
- the orchestrator coordinates the pipeline.

This prevents the project from collapsing into one huge service with mixed responsibilities.

4.2 One Service = One Docker Container

Each service runs inside its own container and therefore has its own:

- Python version;
- packages;
- system libraries;
- CUDA-related dependencies if needed;
- file-level runtime environment.

This is crucial because different workers may use different frameworks such as Paddle, PyTorch, custom OCR stacks, or different local LLMs.

4.3 Stable Contracts Between Services

Services must communicate using stable schemas. A worker can change its internal model later, but its API contract should stay compatible.

This allows:

- easy model replacement;
- cleaner testing;
- easier integration of alternatives;
- less coupling between services.

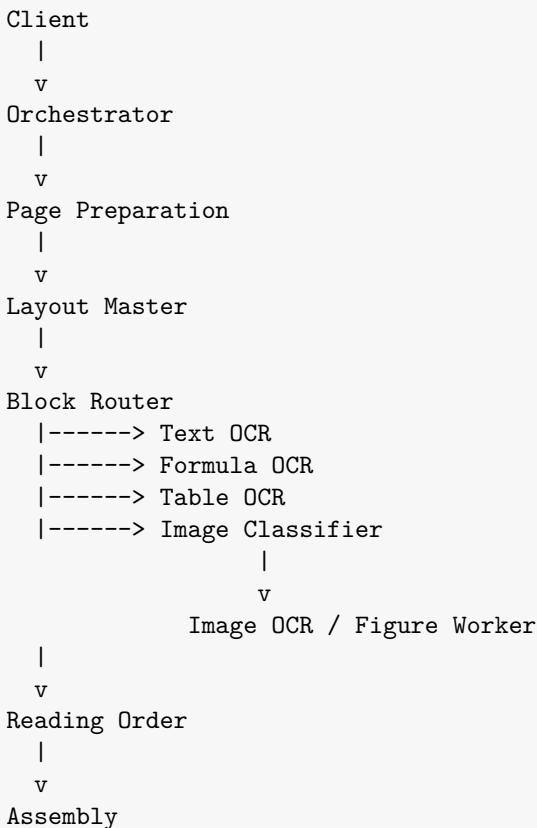
4.4 Artifact-Driven Design

Every pipeline stage should write artifacts to disk. This means the system never hides all intermediate state inside memory only.

This gives:

- reproducibility;
- step-by-step debugging;
- partial reruns;
- easier quality inspection;
- future benchmark datasets;
- operational transparency.

5. High-Level System Scheme



```
|  
v  
Validation  
|  
v  
document.md + document.json + debug artifacts + logs
```

6. Detailed Description of Each Service

6.1 1. Orchestrator

Role: central control plane for a job.

Responsibilities:

- expose public API for upload and status;
- create job ids;
- create job directories;
- call all services in the right order;
- write job metadata and trace files;
- collect service outputs;
- update stage statuses;
- decide final job state.

Must not do:

- real OCR;
- model inference;
- table extraction logic;
- formula recognition;
- figure classification.

Reasoning: the orchestrator should stay focused on pipeline control, not on document intelligence.

6.2 2. Page Preparation Service

Role: normalize PDF input into page-level images.

Responsibilities:

- store original file;
- render PDF pages to PNG or another page-image format;
- create page numbering;
- optionally perform preprocessing:
 - deskew;
 - denoise;
 - contrast normalization;

- resizing.
- write per-page metadata.

Output examples:

- assets/pages/page_0001.png
- assets/pages/page_0002.png
- meta.json

6.3 3. Layout Master

Role: detect logical layout blocks on pages.

Typical block types:

- title;
- text;
- list;
- caption;
- table;
- formula;
- figure / image;
- header;
- footer;
- footnote;
- reference.

Responsibilities:

- run layout detection model;
- output block coordinates and types;
- save raw model output;
- save normalized layout output;
- save debug overlays with boxes and labels.

Important rule: layout service says *what exists and where*. It does not read text and does not assemble Markdown.

6.4 4. Block Router

Role: crop detected blocks and dispatch them to the correct worker.

Responsibilities:

- read normalized layout results;
- crop each block from its page image;
- decide routing target;
- call specialized workers;

- collect worker outputs and save them.

Routing examples:

- text-like block → text OCR;
- formula block → formula OCR;
- table block → table OCR;
- image-like block → image classifier.

6.5 5. Text OCR

Role: process text-like blocks.

Typical inputs:

- paragraphs;
- titles;
- captions;
- list items;
- references;
- headers and footers.

Typical output fields:

- block id;
- recognized text;
- confidence;
- optional formatting hints;
- raw model output.

6.6 6. Formula OCR

Role: process formula blocks and convert them into a useful format such as LaTeX.

Typical output fields:

- block id;
- formula LaTeX;
- confidence;
- raw model response.

Why separate: formulas need different recognition logic than plain OCR text.

6.7 7. Table OCR

Role: turn table crops into structured table data.

Typical output fields:

- block id;
- rows and columns;

- cell coordinates if supported;
- table title if detected;
- markdown-ready representation;
- JSON-ready representation.

Why separate: tables require structure understanding, not only text recognition.

6.8 8. Image Classifier

Role: classify image-like blocks before deeper handling.

Useful labels:

- chart;
- diagram;
- photo;
- illustration;
- scanned-text-image;
- other.

Why this service matters: different image categories need different downstream treatment.

6.9 9. Image OCR / Figure Worker

Role: handle image-like blocks after classification.

Possible tasks:

- OCR inside figures;
- placeholder figure description;
- scientific figure understanding later;
- chart or diagram forwarding later.

6.10 10. Reading Order Service

Role: reconstruct natural human reading order.

Why needed:

- multi-column layouts;
- captions under images;
- sidebars;
- header/footer suppression;
- complex block sequences.

6.11 11. Assembly Service

Role: build final document representations.

Responsibilities:

- merge ordered block outputs;
- generate `document.md`;
- generate `document.json`;
- preserve hierarchy where possible;
- serialize formulas and tables correctly.

6.12 12. Validation Service

Role: verify the final output is complete and coherent.

Responsibilities:

- verify all expected artifacts exist;
- validate schema structure;
- detect suspicious empty blocks;
- mark job status as completed, degraded, or failed.

7. Communication Between Containers

For phase 1, services should communicate through **HTTP on the Docker network**. FastAPI is a very good fit because it gives:

- fast development;
- clean REST endpoints;
- automatic request/response validation;
- interactive docs for testing;
- simple JSON contracts.

7.1 Important Concept

Containers do not communicate because of FastAPI itself. They communicate because they share a Docker network and use HTTP. FastAPI is simply the web framework that exposes the HTTP endpoints.

7.2 Internal Addressing

Inside Docker Compose, services can call each other by service name:

```
http://orchestrator:8000
http://layout_master:8000
http://text_ocr:8000
http://formula_ocr:8000
http://table_ocr:8000
http://image_classifier:8000
```

7.3 Recommended Minimal Endpoints Per Worker

```
GET /health
GET /capabilities
POST /process
```

This keeps every service predictable.

8. Project Repository Structure

A recommended monorepo layout is:

```
pdf-md-pipeline/  
  .gitignore  
  .dockerignore  
  .env.example  
  README.md  
  LICENSE  
  docker-compose.yml  
  docs/  
    ARCHITECTURE.md  
    ARTIFACT_CONTRACT.md  
    API_CONTRACTS.md  
    DEVELOPMENT.md  
    DEPLOYMENT.md  
  configs/  
    orchestrator/  
    page_preparation/  
    layout_master/  
    block_router/  
    text_ocr/  
    formula_ocr/  
    table_ocr/  
    image_classifier/  
    image_ocr/  
    reading_order/  
    assembly/  
    validation/  
  shared/  
    schemas/  
    clients/  
    utils/  
  services/  
    orchestrator/  
    page_preparation/  
    layout_master/  
    block_router/  
    text_ocr/  
    formula_ocr/  
    table_ocr/  
    image_classifier/  
    image_ocr/  
    reading_order/  
    assembly/  
    validation/  
  scripts/  
  data/  
    jobs/  
    models/  
    cache/  
  examples/  
  tests/
```

9. Service Internal Structure

Each service should have a similar internal skeleton to reduce friction:

```
services/text_ocr/  
  app/  
    main.py  
    api/  
    core/  
    models/  
    config.py  
  tests/  
  requirements.txt  
  Dockerfile
```

This consistency matters. It makes the whole monorepo easier to navigate.

10. Artifact Contract

Every job should have a predictable folder structure. Example:

```
data/jobs/job_000001/  
  original/  
    input.pdf  
  preprocessed/  
  assets/  
    pages/  
      page_0001.png  
      page_0002.png  
    layout/  
      page_0001.with_boxes.png  
      page_0001.with_boxes_labeled.png  
    text_blocks/  
    tables/  
    formulas/  
    images/  
  debug/  
    layout_raw.json  
    layout_normalized.json  
    service_responses/  
  meta.json  
  pipeline_trace.json  
  logs.jsonl  
  document.json  
  document.md
```

10.1 Why This Contract Is Important

The artifact contract creates a stable operational truth. Anyone can inspect a job and understand:

- what the input was;
- what each stage produced;
- where a stage failed;
- whether the final result is trustworthy.

11. Job State Model

Every job should keep an explicit state. Suggested stage states:

- pending;
- running;
- completed;
- failed;
- degraded;
- skipped.

Suggested high-level job lifecycle:

```
created
uploaded
preparing_pages
layout_running
layout_done
routing_blocks
text_ocr_running
formula_ocr_running
table_ocr_running
image_classification_running
reading_order_done
assembly_done
validation_done
completed
```

Possible end states:

- completed;
- degraded;
- failed.

12. Docker and Configuration Model

12.1 Why We Still Need Environment Variables

Docker isolates runtime, but it does not remove configuration. Services still need to know:

- where other services live;
- what port to listen on;
- where models are mounted;
- where job artifacts are written;
- CPU or GPU mode;
- timeout and batch settings;
- log level.

12.2 Recommended Files

- `.env.example` goes into Git;
- real `.env` stays local and is not committed.

12.3 Example `.env` Values

```
ORCHESTRATOR_PORT=8000
ARTIFACT_ROOT=/data/jobs
MODEL_ROOT=/models
LAYOUT_MASTER_URL=http://layout_master:8000
TEXT_OCR_URL=http://text_ocr:8000
FORMULA_OCR_URL=http://formula_ocr:8000
TABLE_OCR_URL=http://table_ocr:8000
IMAGE_CLASSIFIER_URL=http://image_classifier:8000
IMAGE_OCR_URL=http://image_ocr:8000
LOG_LEVEL=INFO
```

13. Docker Compose Strategy

For the first version, only the orchestrator should usually expose a host port. Worker services can stay internal.

```
Client browser --> localhost:8000 --> orchestrator
orchestrator --> internal docker network --> workers
```

That keeps the local deployment cleaner and avoids unnecessary host-level port clutter.

Note. As the system grows, different services may need different base images, Python versions, GPU access settings, and volume mounts. This is exactly why one container per service is the right foundation.

14. Python, Virtual Environments, and Containers

A very important conceptual point from the project discussion is this:

- a local `.venv` is optional and mainly useful for development convenience;
- the real runtime environment is each Docker container;
- each container already has its own Python interpreter and its own installed packages.

So the correct mental model is not “one project, one `venv`”.

The correct model is:

```
orchestrator container -> its own Python + deps
layout container       -> its own Python + deps
text OCR container     -> its own Python + deps
formula container      -> its own Python + deps
```

This is what allows different stacks to coexist.

15. Git and GitHub Strategy

15.1 What Must Be Versioned

GitHub should store the durable project definition:

- source code;
- Dockerfiles;
- docker-compose;
- configs;
- docs;
- schemas;
- scripts;
- tiny sample files;
- CI files later;
- `.env.example`.

15.2 What Must Not Be Versioned

GitHub should **not** store:

- huge model weights;
- generated artifacts;
- runtime logs;
- local caches;
- local `.env`;
- `.venv`;
- page images produced during jobs;
- temporary files.

15.3 Recommended .gitignore Sections

```
# Python
__pycache__/
*.pyc
.venv/
.pytest_cache/
.mypy_cache/
.ruff_cache/

# Runtime data
data/jobs/
data/cache/
logs/
tmp/
output/

# Models
models/
```

```
data/models/  
*.safetensors  
*.bin  
*.pt  
*.pth  
*.onnx  
*.pdmodel  
*.pdiparams  
*.ckpt  
  
# IDE / OS  
.vscode/  
.idea/  
.DS_Store  
Thumbs.db  
  
# Secrets  
.env
```

16. Large Models and Multi-Machine Work

A critical operational question in this project is how to work across a laptop, a home PC, or other machines when the system may use huge local LLMs or OCR models.

16.1 What GitHub Is For

GitHub is for:

- code;
- project structure;
- Docker definitions;
- scripts;
- configs;
- documentation.

16.2 What GitHub Is Not For

GitHub is not the right storage layer for hundreds of gigabytes of local model weights. Large local models should live outside the repository.

16.3 Correct Strategy

Use this split:

- **GitHub:** code and architecture;
- **local disks / NAS / model storage:** weights and caches;
- **Docker mounts:** runtime access to those models.

16.4 Per-Machine Example

Laptop:

```
D:\models-lite\
```

Home PC:

```
C:\models\
```

Linux workstation:

```
/srv/models/
```

Each machine clones the same repository, but the local `.env` points containers to that machine's model directory.

16.5 Recommended Operational Pattern

1. clone the same GitHub repo on each machine;
2. create local `.env`;
3. define machine-specific model paths;
4. mount model directories into containers;
5. run docker-compose locally.

16.6 Model Setup Scripts

The repository should include scripts such as:

- `scripts/fetch-models.sh`
- `scripts/fetch-models.ps1`

These scripts can:

- download approved models from their official source;
- copy models from a NAS or shared location;
- verify checksums;
- place files into expected folders.

That is far better than trying to store giant binary weights in Git.

17. Enumeration of Main Instruments and Technologies

This section lists the main instruments the project will use or is designed to support.

17.1 Infrastructure and Packaging

- **Git** – local version control.
- **GitHub** – remote code repository and collaboration hub.
- **Docker** – runtime isolation per service.
- **Docker Compose** – local multi-service orchestration.

17.2 Programming and API Layer

- **Python** – primary implementation language.
- **FastAPI** – service HTTP APIs.
- **Pydantic** – schema validation.
- **Uvicorn** – ASGI server for services.

17.3 Shared Engineering Tools

- **pytest** – tests.
- **ruff / mypy** – code quality and typing later.
- **logging / json logs** – observability.

17.4 Possible Document / Vision Engines

This architecture is intentionally engine-agnostic. Different workers can later use different stacks, for example:

- layout model service;
- OCR model service;
- formula-recognition model service;
- table extraction service;
- local vision-language model for image classification.

The architecture cares more about **contracts** than about locking the project to one model.

18. Suggested API Contracts

18.1 Health

```
GET /health
```

Response:

```
{
  "status": "ok"
}
```

18.2 Capabilities

```
GET /capabilities
```

Response:

```
{
  "service": "text_ocr",
  "version": "0.1.0",
  "mode": "placeholder",
  "supported_inputs": ["image_crop"]
}
```

18.3 Process

```
POST /process
```

```
Request:
```

```
{
  "job_id": "job_000001",
  "block_id": "p1_b7",
  "input_path": "/data/jobs/job_000001/assets/text_blocks/p1_b7.png"
}
```

```
Response:
```

```
{
  "job_id": "job_000001",
  "block_id": "p1_b7",
  "status": "completed",
  "output_path": "/data/jobs/job_000001/assets/text_blocks/p1_b7.result.json"
}
```

19. Suggested Shared Schemas

The `shared/schemas` package should define canonical structures such as:

- Job
- StageStatus
- LayoutBlock
- OCRResult
- FormulaResult
- TableResult
- ImageClassificationResult
- ArtifactRecord

This keeps all services speaking the same language.

20. Ready Build Plan

This section is the practical plan you can follow almost directly while building the project.

20.1 Stage 0: Create the Repository

1. Create a new Git repository.
2. Create the monorepo folder tree.
3. Add `README.md`, `LICENSE`, `.gitignore`, `.dockerignore`, and `.env.example`.
4. Commit the empty structure first.

20.2 Stage 1: Create Placeholder Services

For each planned service:

1. create `app/main.py`;

2. add `requirements.txt`;
3. add a service-specific `Dockerfile`;
4. implement:
 - `/health`
 - `/capabilities`
 - `/process`
5. return placeholder JSON from `/process`.

20.3 Stage 2: Create the Orchestrator

1. create upload endpoint;
2. generate `job_id`;
3. create the artifact folder tree;
4. call page preparation service;
5. call layout service;
6. for now call placeholder workers;
7. write a placeholder `document.md`;
8. expose job status endpoint.

20.4 Stage 3: Add docker-compose

1. define one service per container;
2. expose only orchestrator to host;
3. mount:
 - artifact root;
 - model directories;
 - optional config volumes.
4. verify orchestrator can reach workers by service name.

20.5 Stage 4: Add Shared Schemas

1. define Pydantic models for job, stage, block, and result types;
2. use these schemas in request and response bodies;
3. centralize status enums and artifact metadata.

20.6 Stage 5: Add Basic Integration Tests

1. health checks for all services;
2. orchestrator pipeline smoke test;
3. verify placeholder output files exist;
4. verify `document.md` and `document.json` are generated.

20.7 Stage 6: Replace Placeholders One by One

1. first replace page preparation with real PDF rendering;
2. then replace layout;
3. then text OCR;
4. then formula OCR;
5. then table OCR;
6. then image classification;
7. then reading order and validation improvements.

20.8 Stage 7: Production Hardening Later

1. retries;
2. degraded-mode logic;
3. queue-based execution if needed;
4. better metrics;
5. distributed workers;
6. web UI or admin panel later.

21. Recommended Reading Order for Building the Project

If you want to study the project and build it only from this plan, use this order:

1. read the high-level architecture section;
2. read the service descriptions;
3. read the repository structure;
4. read the artifact contract;
5. read the Docker and configuration model;
6. read the Git and model-storage strategy;
7. follow the ready build plan stage by stage;
8. only then integrate real models.

That order will keep the project understandable.

22. Final Architecture Summary

The project should be understood as this:

- **GitHub repository** = code, docs, configs, Dockerfiles, schemas.
- **Docker Compose** = starts the whole platform locally.
- **Each service** = one container, one responsibility, one runtime.
- **FastAPI** = simple HTTP communication layer.
- **Artifact folders** = persistent operational truth of each job.

- **Local model folders** = heavy weights outside Git.

This design is the right base for a system that starts as a clean placeholder scaffold and later grows into a serious offline PDF-to-Markdown platform.

23. Concise Master Scheme

